

kickstart_lecture2_python-syntax-II-enhanced

November 14, 2024

1 Python Kickstart Lecture 2: Syntax - Control Structures and Functions

1.1 Control Structures: If... else...

```
[1]: # Define a variable to check against in the conditional structure  
check_var = 45  
  
# if  
  
check_var = 45
```

```
[2]: # Conditional statement to check if check_var equals 45  
# If true, increment check_var by 10, calculate its square, and print the result  
if check_var == 45:  
    check_var += 10  
    check_var_square = check_var ** 2  
    print(check_var_square)
```

3025

```
[3]: 55**2
```

```
[3]: 3025
```

```
[4]: check_var
```

```
[4]: 55
```

```
[5]: # Another example of conditional structure with an else clause  
check_var = 45  
if check_var == 45:  
    check_var += 10  
    check_var_square = check_var ** 2  
    print(check_var_square)  
else:  
    print('check_var does not equal 45')
```

3025

1.2 Control Structures: For and While Loops

```
[6]: # Using a "while" loop to print powers of 2 for each j in range(10)
i = 0
while i<10:
    result = 2**i
    print(i, result)
    i+=1
```

```
0 1
1 2
2 4
3 8
4 16
5 32
6 64
7 128
8 256
9 512
```

```
[7]: # The same with a "for" loop
for j in range(10):
    result = 2**j
    print(j,result)
```

```
0 1
1 2
2 4
3 8
4 16
5 32
6 64
7 128
8 256
9 512
```

1.3 Functions

```
[8]: # Function that calculates and prints the cube of the number 5
def cube_five_function():
    number = 5
    cube_of_number = number ** 3
    print('Cube of 5 is:', cube_of_number)
```

```
[9]: # Call the cube_five_function to display the cube of 5
cube_five_function()
```

```
Cube of 5 is: 125
```

```
[10]: # Function that accepts a number as input and prints its cube
def cube_a_number_function(number):
    cube_of_number = number ** 3
    print(f'Cube of {number} is:', cube_of_number)
```

```
[11]: # Call the cube_a_number_function with 100 as input
cube_a_number_function(100)
```

Cube of 100 is: 1000000

```
[12]: # Capture the result of cube_a_number_function with 100 as input
# but note it returns None
res = cube_a_number_function(100)
```

Cube of 100 is: 1000000

```
[13]: # Display the value of res (will be None)
# since cube_a_number_function does not return a value
res
```

```
[14]: # Add "return" values
def cube_a_number_function_advanced(number):
    cube_of_number = number**3
    print(cube_of_number)
    return cube_of_number
```

```
[15]: # Call the advanced cube function and store the result
res2 = cube_a_number_function_advanced(100)
res2
```

1000000

```
[15]: 1000000
```

```
[16]: # Attempt to call the advanced cube function without arguments,
# expected to raise an error
cube_a_number_function_advanced()
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[16], line 3
      1 # Attempt to call the advanced cube function without arguments,
      2 # expected to raise an error
----> 3 cube_a_number_function_advanced()

TypeError: cube_a_number_function_advanced() missing 1 required positional
argument: 'number'
```

```
[17]: # Assign defaults
def cube_a_number_function_advanced2(number=1):
    cube_of_number = number**3
    print(cube_of_number)
    return cube_of_number
```

```
[18]: # Call the advanced cube function with a default argument, calculating the cube
      ↪ of 5
cube_a_number_function_advanced2(5)
```

125

[18]: 125

```
[ ]:
```